ECE 445L Lab 10

Final Design and Evaluation of an Embedded System

This laboratory assignment accompanies the book, *Embedded Systems: Real-Time Interfacing to ARM Cortex M Microcontrollers, ISBN-13: 978-1463590154*, by Jonathan W. Valvano, copyright © 2026.

Table of Contents

Table of Contents 1
Team Size 2
Goals 2
Starter Files 2
Required Hardware 2
Lab Overview 2
Preparation 2
Procedure 2
Lab Checkout 4
Lab Report 4
  Deliverable 1 4
  Deliverable 2 4
  Deliverable 3 4
  Deliverable 4 4
  Deliverable 5 4
Hints 5
  Procedure for bring up a stand-alone MSPM0, including XDS110 Connection 5

Team Size

The team size for this lab is **4**.

Goals

- Build, test, and validate an embedded system.

Starter Files

- Starter project:
    - Use the hardware and software developed in Lab 7A and Lab7B.

Required Hardware

There is no required hardware for this lab outside of the restrictions for the final competition provided further in the document.

Lab Overview

You will complete the embedded system you began in Labs 7A and 7B. This lab includes hardware construction, software debugging, and system evaluation. There will be a "science fair" style public demonstration at the end of Lab 10. The TAs shall judge your project during your lab demo for qualification for the final competition, and the professor(s) shall present special awards to the teams with the best designs at the end of the competition. The best designs are selected by your fellow students and observers.

Preparation

1. **Revise your requirements document** one final time to reflect the final version of your design. Emphasize in detail how your system will be evaluated, tested, and verified.
2. **Gather any remaining parts** needed to build your system, including any makeshift repairs.
3. **Update your Bill of Materials** (BoM).
4. **Design and implement the high-level main application** that demonstrates the final objective of your embedded system. If you want to change the project scope, please consult with the TA for approval.

Procedure

The following is a procedure for assembling and debugging your system. Feel free to adjust this to what makes sense for your circuit layout.

1. Verify that the Lab 7A breadboard system works. Using integration tests or other tests, understand where corrections need to be made to your PCB preemptively to get it in the same working state as your breadboard prototype.

2. Verify that there are no shorts between the various power and ground lines. Ensure that:
Any power signal (e.g. 3V3, 5V0, VBATT, GND) does not short to any other signal.
Shorts can occur. So, before soldering, check for shorts caused by design or manufacturing

3. Solder down the power regulation circuitry, including:
Power input connectors.

On-off switch.
Regulators.
Regulator capacitors and supporting passives.

Check for shorts from power to ground. Verify that when power is connected, the corresponding output voltages are generated.

4. Solder down the LaunchPad headers. Verify that when plugged in that LaunchPad turns on. See additional instructions for assembling the SMD chip of the TM4C.
Check for shorts from power to ground. Verify that when power is connected, the corresponding output voltages are generated.

5. Solder down debug components. Solder down all test points, logic analyzer headers, and things like power supply LEDs and heartbeat LEDs.
Check for shorts from power to ground. Verify that when power is connected, the corresponding output voltages are generated.

6. Solder down each subcircuit.
Solder down components in subcircuit in order of height – lowest ones first.
Check for shorts from power to ground. Verify that when power is connected, the corresponding output voltages are generated.
Verify subcircuit functionality by using your unit/integration tests.
Measure current consumption for this subcircuit.
At this point, everything should have been soldered down and verified individually.

7. Debug your embedded system application.

Lab Checkout

You should be able to demonstrate the full functionality of your system as described in your requirements document. Present your BoM, project cost, and features for approval for the final competition. Be prepared to discuss any difficulties and challenges during the implementation, and any testing procedures or risk mitigation processes.

Lab Report

Deliverable 1

Final requirements document including the data flow graph and call graph.

Deliverable 2

Describe the software tests that you added since Lab 7 to verify your modules and their integration.

Deliverable 3

Describe the hardware tests that you performed since Lab 7 to verify your embedded system.

Deliverable 4

Measurements of the current for various system states as appropriate for your design. E.g., sleep, idle, receiving, transmitting, and/or active.

Deliverable 5

Validate the performance of your embedded system. For each aspect of the system you are verifying, you must create a 1-3 sentence paragraph outlining:

- How you performed this measurement
- How/if the measured behavior meets your system's requirements
- And what sources of errors may affect your measurement

Validate at least two performance metrics of your system from the following list:

- CPU utilization (thread profile) measured separately for each module.
    - Maximum execution time for each ISR
    - Maximum time doing work in the main loop
- Ping latency (Wi-Fi, Bluetooth, LoRa, 433Mhz Radio, etc.)
- DAC or ADC sampling jitter
- Signal to noise ratio (SNR)
- Used vs total bandwidth of IC-to-IC communication protocols (UART, SPI, I2C, CAN, etc.)
- Feel free to add a performance metric you think is relevant to your project

Hints

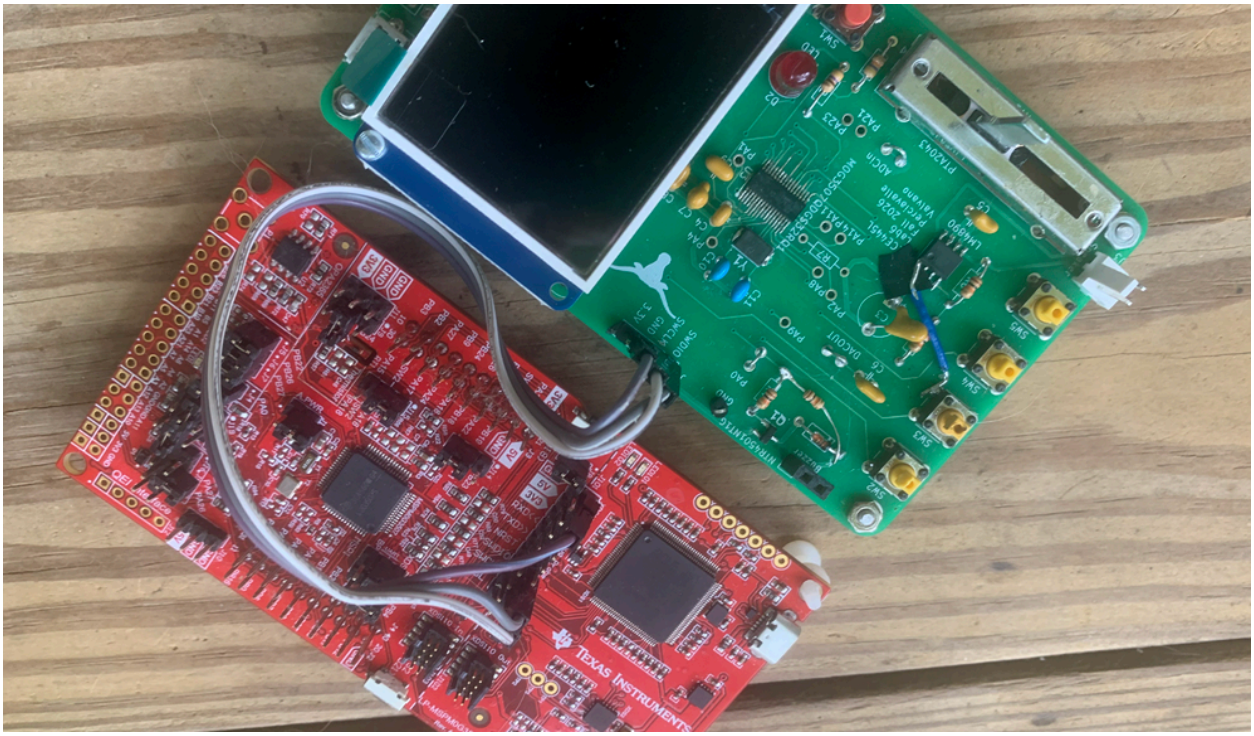
1. When you pick up your PCB boards, make sure you get your board.
2. You should have collected all the electrical parts during Lab 7A. Think about your system mechanically. Here in Lab 10 collect nuts/bolts/screws. Will it all fit together? Put it together before soldering. This will help to determine the proper order of construction.
3. Take the system with you to job interviews.

Procedure for bring up a stand-alone MSPM0, including XDS110 Connection

Follow these steps carefully.

1. Before you start, verify there are no shorts between +3.3V power, ground, battery input, or 5V power line. Solder the power connectors and regulators first. Before soldering the 3.3V regulator, please put together and test your regulator circuit(s) to verify you have connected the power rails correctly. Now proceed to solder the regulator and two caps to the PCB from your breadboard (everything you need to create 3.3V power). Place tantalums in polarized direction. Include 3.3V and ground test points.
2. Test the power. Connect voltmeter(s) to the regulated power line(s) 3.3V and 5V if present. Use a bench supply and limit the input current to 50 mA. Apply power to your board (remove power immediately if not 3.3V).
3. Clean the MSPM0 pads with isopropyl alcohol. Align the MSPM0 on the board. Apply solder flux to the pin. Align the MSPM0 on the board again. Gently touch the solder iron then the solder ONE pin. You use VERY little solder. Align the MSPM0 on the board by reflowing the solder on the ONE pin. Show it to a TA/professor or Dan Book. This is critical. Solder a second pin and again have someone check it. Then solder the rest of the 32 pins. Clean solder-bridges with solder-removal braid.
4. Verify there are no shorts between +3.3V power, ground, battery input, or 5V power line. Check for shorts between adjacent pins on the processor. Use thin wire (28 or 30 gauge) to touch the ohmmeter probes to processor pins. If you can see excessive solder between pins, remove it with solder wick. If adjacent pins are connected but you see no excessive solder, very gently cut between processor pins with a sharp knife.
5. Solder crystal and all caps around the processor. Make sure crystal caps are 10 pF ceramic C0G, and double check all values. For example, ceramic 104 means $10*10^4$pF= 0.1uF. Verify there are no shorts between +3.3V power, ground, battery input, or 5V power line.
6. Solder the debug connector, reset button, and pull up. Resistor pulldown on PA2 is needed only if you are running without a crystal. Verify there are no shorts between +3.3V power, ground, battery input, or 5V power line.
7. Connect voltmeters to all battery and 3.3V power signals. Using a bench power supply, set the bench supply current limit to 100mA to ensure safe operation and prevent damage. Make sure to test with a multi-meter to get the polarity right to not damage the processor. The current should be 40 to 80mA. Turn it off immediately if the voltages are not correct or if the current is above 100 mA.
8. Remove power from both your target system and an MSPM0 LaunchPad. Remove all 9 jumpers between the XDS110 debugger and MSPM0 on the LaunchPad. Connect these three signals between the XDS110 debugger and your target (Do not connect 3.3V)

GND	to	GND
SWCLK	to	SWCLK
SWDIO	to	SWDIO



Power the XDS110 debugger by connecting the USB cable to your laptop and power your target in the same way as step 7. Start with an empty project like **Scratch**. You should be able to program your target in the usual way. If you run a project that calls **Clock_Init_HFXT_16_80MHz(0)** you will see sine waves on both PA5 and PA6 (both sides of the 16 MHz crystal). Assuming you have not used PA14 for your system, you can also run a project that calls **Clock_Init_HFXT_16_80MHz(1)**, and you will see also see a 10 MHz sine wave on PA14. These clock options are available

Clock_Init_HFXT_16_80MHz(0) runs at 80 MHz 0.005% accuracy with the 16 MHz crystal.
Clock_Init80MHz(0) runs at 80 MHz 1% accuracy without the crystal.
Clock_Init_HFXT_16MHz() runs at 16 MHz 0.005% accuracy with the 16 MHz crystal
Clock_Init(OSCFREQ4MHZ) runs at 4 MHz 1% accuracy without the crystal
Clock_Init(OSCFREQ32MHZ) runs at 32 MHz 1% accuracy without the crystal

Do not call the following two functions because they assume you have a 40 MHz crystal

Clock_Init_HFXT_40_80MHz
Clock_Init40MHz

9. Solder components in order of height and in a modular way. The components with the lowest height come first. Check for shorts from power to ground before applying power again. Run very simple test code to check each module separately. If your system has both 3.3V and 5V power, please place a voltmeter on the 5V output and measure the current while testing it for the first time. Record the current required to run the entire PCB.